Interaction and Integration Specification

What is defined, how the pieces connect, and what comes next

Jonathan Ma

October 2026

Why we moved into a specification phase

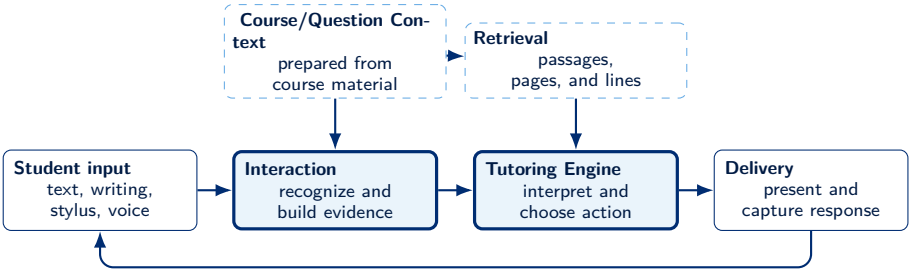
The integration problem

We have working ideas for course ingestion, retrieval, multimodal interaction, tutoring decisions, and delivery. The next risk is that each part assumes different inputs, outputs, or ownership.

Define	Connect	Review
State what each component is responsible for and what it must not do.	Give every handoff a producer, consumer, record, status, and failure path.	Resolve disagreements in Markdown before they become incompatible code.

Current goal: create an implementable system contract, then build against it.

The proposed tutoring loop



Each response becomes new evidence. The system reassesses and chooses the next action instead of treating messages as isolated questions.

My module: the interaction boundary

What it owns

- Capture and preserve student submissions
- Route handwriting and voice recognition
- Confirm meaning-changing ambiguity
- Build factual Student Evidence
- Link attempts, help, and revisions
- Coordinate turn handoffs and status

What stays downstream

- Mathematical correctness judgments
- Misconception or mastery diagnosis
- Course-passage ranking
- Selection of hints or explanations
- Avatar and channel presentation

Boundary rule: preserve what the student did; let the Tutoring Engine decide what it means.

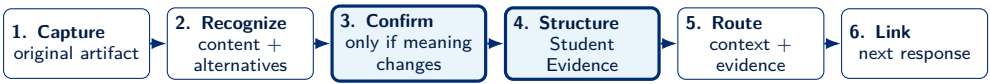
Practical V1 input scope

Input	V1	Approach
Typed text and TeX	Include	Baseline input with original source preserved separately from rendering.
Handwritten image/PDF	Include	One visual-recognition path with confirmation for important ambiguity.
Direct stylus input	Include	Export the canvas and reuse the handwriting path.
Voice	Simplify	Record one audio clip, transcribe it, and allow confirmation or editing.
Live voice/video/avatar	Defer	Add after the asynchronous tutoring loop and delivery contracts are stable.

Why this scope

It tests the evidence and handoff design across realistic modalities without requiring multiple real-time systems at once.

From a submission to Student Evidence



Evidence records facts

Original work, confirmed transcription, observable steps, final answer, explanation, stated uncertainty, prior help, and revision links.

Learner State interprets facts

What appears understood, what remains unresolved, and which gap or misconception is plausible.

Ambiguity: confirm meaning, not every character

Example

A handwritten expression could be read as:

$$x_1 + x_2 = 1$$

or

$$x_l + x_2 = l$$

Expected behavior

1. Preserve the original image
2. Store both recognition candidates
3. Show the relevant crop to the student
4. Ask for a selection or free-form edit
5. Log raw and corrected values separately
6. Mark unresolved evidence as unconfirmed

The recognizer must not silently change an incorrect student equation into a correct one.

The shared records connect the modules

Record	Handoff	Purpose
`CourseContext` and `QuestionContext`	Ingestion → system	Identify the course, question, objectives, rules, and approved source versions.
`StudentEvidence`	Interaction → Tutoring Engine	Preserve the student's confirmed work and its history.
`RetrievalRequest` / `GroundingBundle`	System ↔ Retrieval	Ask for and return relevant course passages.
`LearnerState` / `TutoringAction`	Tutoring Engine → Delivery	Explain the current interpretation and prescribe one next action.
`DeliveryResult`	Delivery → Interaction	Record what was actually presented and continue the turn.

The draft interface specification defines required IDs, status, provenance, validation, and failure behavior for each record.

Grounding and logging are part of the contract

Precise course grounding

- Store a passage, page, and normalized line range
- Retain document and content-version identifiers
- Provide an authorized highlight link
- Return `no_support_found` instead of inventing a citation

Configurable logging

- Separate required session state from diagnostics
- Allow optional logging to be turned on or off
- Keep raw and corrected recognition distinct
- Record versions, status, latency, retries, and cost when available

Traceability target: connect a tutoring action to the evidence, course passage, and component versions that produced it.

What is complete in the first draft

Shared specification

- Repository structure and module template
- High-level architecture and tutoring workflow
- Initial shared-record definitions
- Logging, correction, grounding, and failure rules

Interaction specification

- V1 modality recommendation
- Recognition and ambiguity behavior
- Student Evidence and revision links
- Handoffs to context, retrieval, tutoring, and delivery
- Acceptance scenarios and open questions

Status: reviewable V0.1 contracts, not a frozen architecture or completed implementation.

What teammates are reviewing now

Ovunc	Sean	Yining
Course and Question Context, ingestion behavior, source versions, normalized pages, and stable line mappings.	Retrieval requests, chunk/index versions, passage ranking, page/line citations, and highlight links.	Tutoring Action requirements, avatar/channel presentation, Delivery Result, and return of the next response.

The purpose of review

Each teammate should change the shared interfaces when their module cannot reliably produce or consume what the current draft assumes.

Next: resolve contracts, then implement one slice

Decisions to resolve

- Who creates the final retrieval request?
- Which Question Context fields are guaranteed?
- How do page/line mappings remain stable?
- What evidence is sufficient for Learner State?
- Where does recognition confirmation end and tutoring begin?

Implementation sequence

1. Reconcile teammate module specifications
2. Finalize Tutoring Engine and Delivery contracts
3. Freeze interface version 0.1
4. Implement one text-first tutoring loop
5. Add handwriting, stylus, and recorded voice paths
6. Validate with a Calculus scenario set and independent math review